

O Especialista em Automação e o Líder de QA "Escaldado"

Transcrição da prática 1 | 8 de set. de 2026 8:52

Vágner Lins 00:00

Olá! Olha, eu não quero saber de IA 'mágica' que escreve código e joga no meu repositório sem eu saber se aquilo sequer compila. Tivemos problemas sérios semana passada. Se esse Claude Code vai morar no terminal dos meus devs, ele tem que seguir as minhas regras.

Me explica o básico: O que são esses 'Hooks' e como eu uso isso para forçar a IA a rodar minha suíte de testes inteira automaticamente antes de ela me dizer que terminou um bugfix? Eu consigo travar a ferramenta se os testes falha

Você 01:22

Hooks são gatilhos automatizados executados quando um evento específico acontece em um sistema (como antes de um commit no Git ou ao receber um webhook do GitHub). No contexto de desenvolvimento com assistentes de IA (como agentes autônomos, Devin, Cursor ou ferramentas de CLI como o Aider), você pode configurar hooks de pré-execução ou pós-processamento para garantir que a IA não dê a tarefa como concluída até que o comando de teste retorne sucesso. Sim, você consegue travar a ferramenta se os testes falharem. A maioria das ferramentas de IA e automação dependem do exit code (código de saída) do seu terminal: se o comando de testes falhar (retornando algo diferente de 0), o hook interrompe o fluxo da IA, impedindo-a de commitar ou marcar a tarefa como pronta.

Vágner Lins 01:24

Hmm, entendi o conceito. Então, se eu entendi direito, esse hook seria como um porteiro no processo, certo? Ele só deixa a IA continuar se o 'código de saída' for positivo, ou seja, se os testes passarem. Agora, fala sério: consigo configurar isso de forma que seja impossível de ignorar? Tipo, nenhum dev consegue dar um "bypass" ou "forçar" o commit mesmo se falhar?

Você 02:57

Exatamente! A analogia do porteiro é perfeita. Se o código de saída for 0 (sucesso), ele abre a porta; se for qualquer outra coisa (erro), ele barra a entrada. Respondendo direto à sua pergunta: Localmente na máquina do desenvolvedor, é impossível criar um hook 100% inviolável. Qualquer dev com acesso ao terminal consegue burlar os hooks locais usando comandos como `git commit --no-verify` (que ignora os git hooks) ou simplesmente desativando as extensões da IDE. Para tornar o bloqueio completamente impossível de ignorar, você precisa transferir essa validação da máquina do desenvolvedor para o servidor onde o código fica guardado. É aqui que entra o CI/CD (Integração Contínua) e as Regras de Proteção de Branch.

Vágner Lins 03:00

Ah, agora estamos falando a minha língua! Então, basicamente, os hooks locais são um filtro inicial, mas o verdadeiro bloqueio está no servidor, com o CI/CD e as regras de proteção de branch garantindo que nada passa sem validação. Beleza, dá para configurar isso para que qualquer código enviado para a branch principal só seja aceito se passar por toda a suíte de testes? E como eu monitoro se os devs estão jogando limpo nos hooks locais antes disso?

Você 05:03

Para garantir que a branch main só aceite códigos aprovados pelos testes, você deve usar as Branch Protection Rules (Regras de Proteção de Branch). Vá até o repositório do seu projeto no GitHub. Clique em Settings (Configurações) > Branches. Em Branch protection rules, clique em Add rule (ou Edit na regra da sua branch principal, ex: main). Ative as seguintes opções cruciais: Require a pull request before merging: Ninguém (nem a IA) poderá dar git push direto na main. Tudo precisará passar por um Pull Request (PR). Require status checks to pass before merging: Ative esta opção e, na barra de pesquisa que aparecer abaixo, digite o nome do seu job do CI/CD (ex: run-tests ou o nome do seu workflow do GitHub Actions). Enforce administrative: Ative isso se quiser que nem você (o administrador) consiga burlar a regra. Clique em Save changes. Pronto. A partir desse momento, se o teste falhar no servidor do GitHub, o botão de merge fica completamente travado e ninguém consegue colocar o código para dentro.

Vágner Lins 05:05

Perfeito, isso é exatamente o que eu precisava ouvir! Então, com essas regras de proteção e os PRs obrigatórios, nem a IA, nem os devs conseguem colocar código quebrado na main. Agora, última dúvida: e se alguém tentar passar pelos hooks locais na máquina e só confiar no CI/CD? Como eu crio uma cultura em que os devs realmente rodem os testes antes de subir qualquer coisa?

O Especialista em Automação e o Líder de QA "Escaldado"

Você 06:17

Criar essa cultura exige equilibrar duas frentes: facilitar a vida do desenvolvedor (para que ele não queira pular o hook) e dar visibilidade aos abusos (para que o time perceba que pular o processo gera retrabalho). Se os desenvolvedores estão ignorando os testes locais para confiar apenas no CI/CD, geralmente é por um destes três motivos: os testes locais demoram muito, os hooks quebram por motivos bobos (como formatação de texto) ou eles simplesmente não entenderam o impacto disso no time.

Vágner Lins 06:19

Boa análise! Então, pra resolver, eu precisaria otimizar os testes pra rodarem rapidinho, ajustar os hooks pra não travarem por besteira, e, claro, educar o time sobre o impacto de pular etapas. Faz sentido. Talvez até implementar algo como um relatório semanal mostrando os commits que precisaram ser revertidos por quebra de testes no CI/CD, né? Isso pode dar uma boa sacudida no pessoal.

Você 07:49

Para configurar o Claude (ou ferramentas baseadas na API da Anthropic, como o Cursor, Aider ou scripts personalizados de agentes de IA) para rodar um linter ou scan de segurança antes de dar uma tarefa por encerrada, nós usamos o conceito de Gatilhos de Pré-Execução baseados em Regras de Sistema. Como o Claude não roda diretamente dentro da sua máquina por conta própria (ele precisa de uma interface/ferramenta intermediária que execute os comandos de terminal), a configuração é feita instruindo a IA através do arquivo de regras do projeto ou usando os parâmetros da CLI que você utiliza.

Vágner Lins 07:52

Entendi, então eu preciso configurar essas regras ou parâmetros pra que o Claude execute o linter ou o scan de segurança antes de considerar a tarefa finalizada. Isso me dá controle sobre o que ele faz, mas como eu garanto que essas regras não sejam alteradas ou ignoradas por descuido ou má intenção? Tem como travar isso também?

Você 08:54

Sim, tem como travar completamente. Assim como vimos na proteção dos testes, a regra de ouro da segurança em engenharia de software se aplica aqui: nunca confie no que acontece apenas na máquina local ou na interface da IA. Se um desenvolvedor alterar o arquivo `.cursorrules` para remover as travas, ou se esquecer de passar as flags de segurança no terminal ao rodar o Claude, a barreira local cai. Para tornar essas regras de linter e segurança invioláveis, você precisa aplicar uma estratégia de dupla camada.

Vágner Lins 08:57

Certo, entendi que a camada local é vulnerável e precisa de reforço. Essa dupla camada seria algo como configurar validações no servidor ou no pipeline do CI/CD também, garantindo que as regras sejam verificadas novamente antes de qualquer código entrar no repositório? Assim eu teria certeza de que mesmo se alguém burlar localmente, o sistema pega no servidor, né?

Você 09:54

Exatamente! Você pescou perfeitamente a essência da segurança moderna em desenvolvimento. Essa estratégia se chama Defesa em Camadas (ou Defense in Depth). O seu raciocínio está 100% correto: a validação local (com o Claude, hooks e linters na máquina do dev) serve para dar velocidade e feedback instantâneo, enquanto o CI/CD no servidor funciona como a Suprema Corte do seu código. Se passar algo quebrado localmente — seja por malícia, erro de configuração ou pressa —, o servidor pega, barra o código e impede que ele contamine a sua branch principal (main ou master).

Vágner Lins 09:56

Show de bola! Defesa em Camadas é exatamente o tipo de abordagem que me deixa tranquilo. Com essas validações locais e no servidor, dá pra confiar que o Claude vai trabalhar dentro das regras e sem comprometer a qualidade. Vou seguir essa linha e ajustar tudo pra evitar qualquer deslize. Valeu pelas explicações, ficou bem claro agora!

Fim da sessão